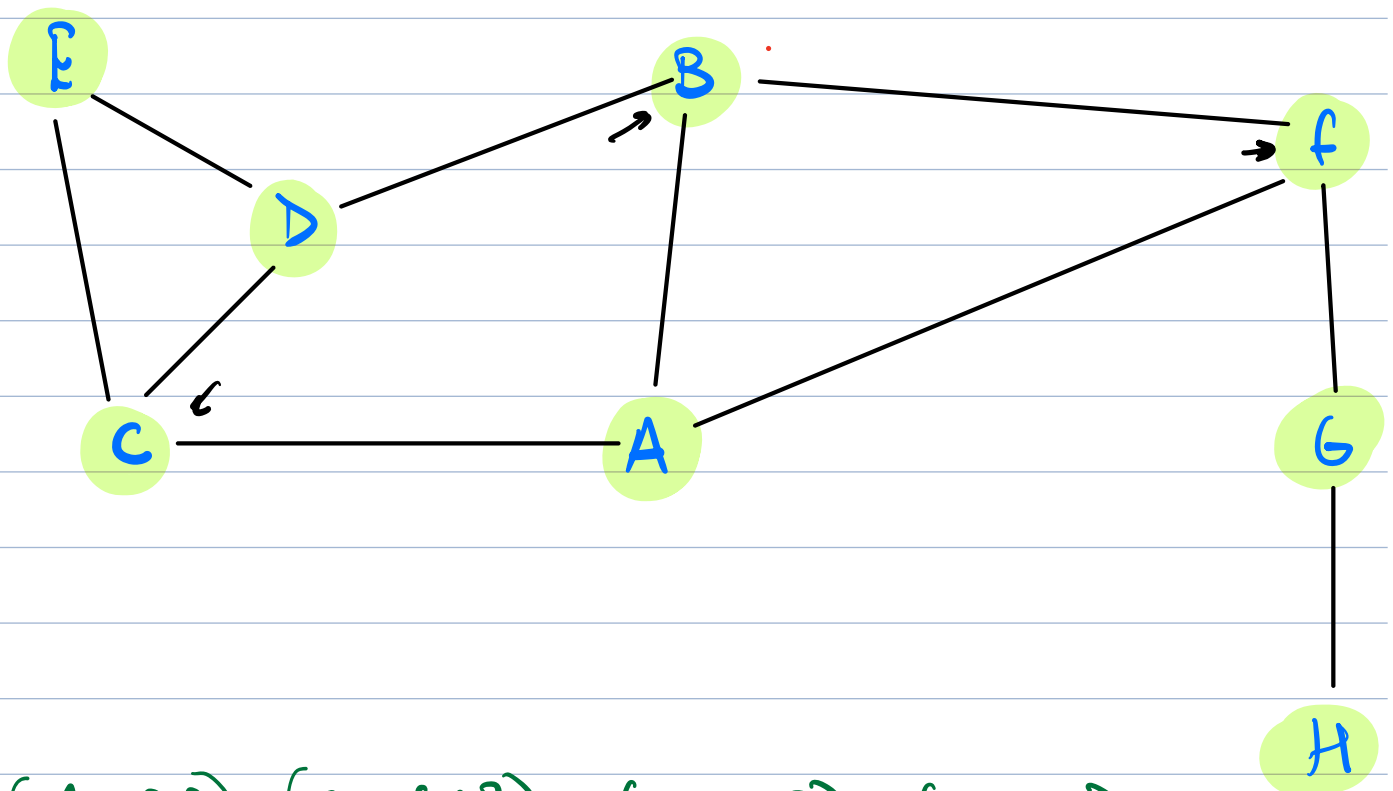


BFS With Path Trace



~~$(A, [])$~~ , ~~$(B, [A])$~~ , ~~$(C, [A])$~~ , ~~$(F, [A])$~~

~~$(D, [A, B])$~~ , ~~$(E, [A, C])$~~ , ~~$(G, [A, F])$~~ ,
 $(H, [A, F, G])$

$(B, [A])$, $(C, [A])$

$(F, [A])$, $(G, [A, F])$

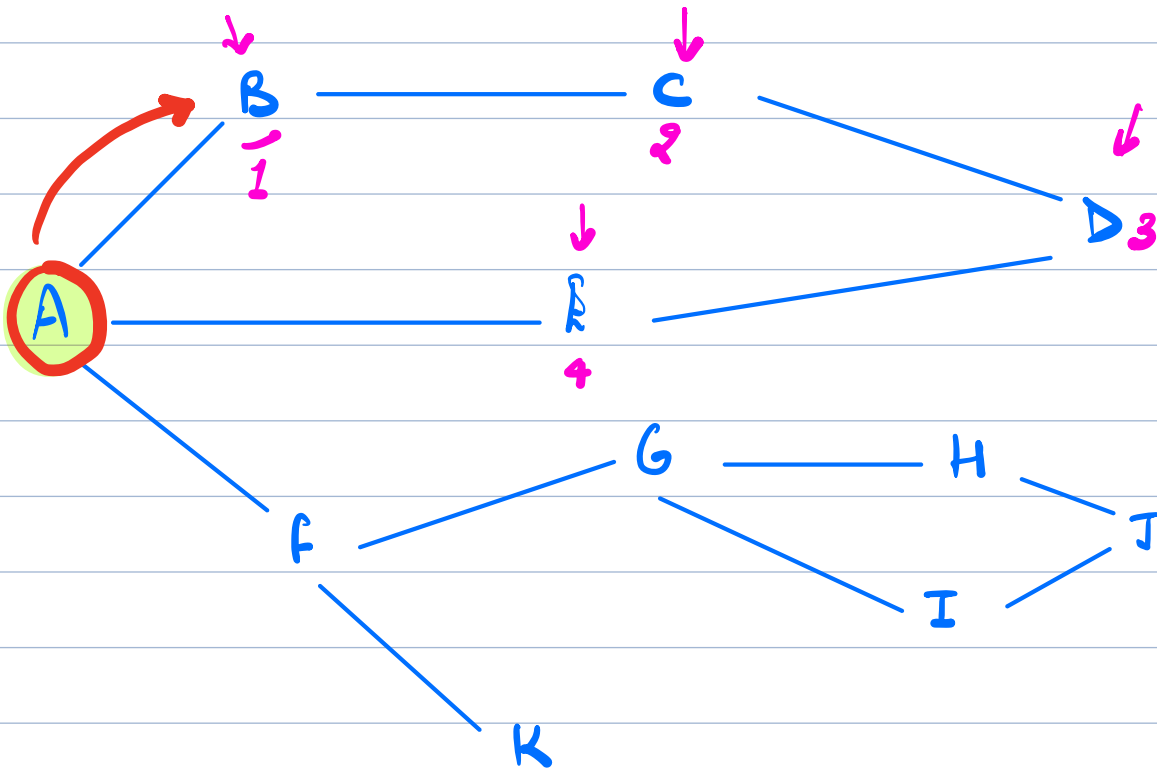
Dfs (Depth first Search)

Code

```
void dfs(source) {  
    vst.insert(source);
```

```
    for (all Node u connected to source) {  
        if (!vst.contains(u)) {  
            dfs(u);  
        }  
    }  
}
```

T.C. = $O(V+E)$



Code

bool isCyclePresent (src, ^{NULL} parent) {

vst.insert (src);

for (all Node u connected to src) {
 if (vst.contains (u) && u != parent)
 return true;

}

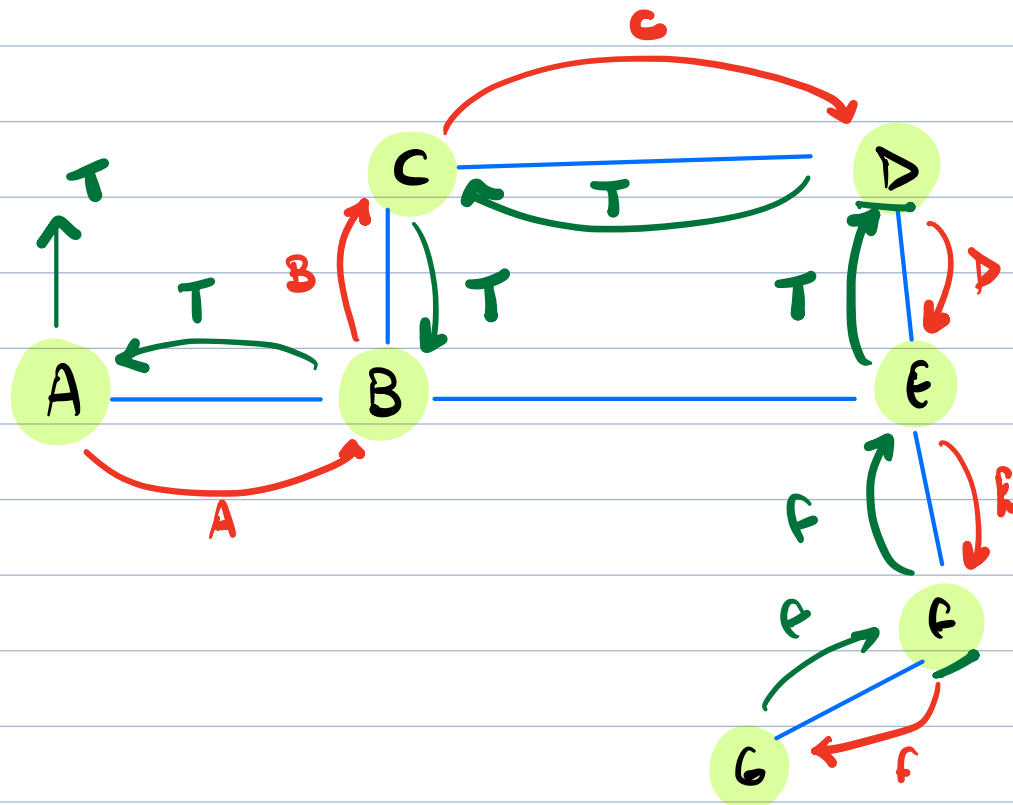
if (! vst.contains (u) && isCyclePresent (u, src))
 return true;

}

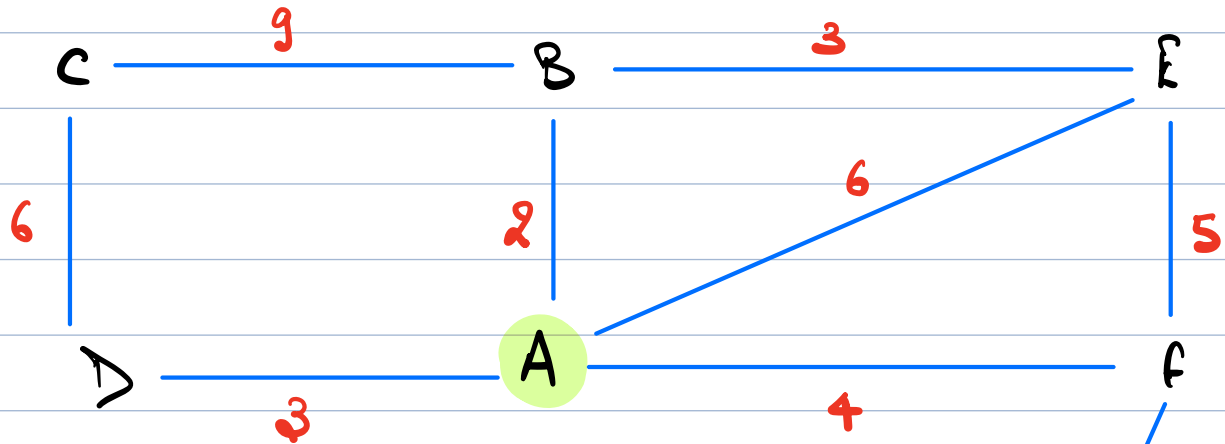
}

return false;

T.C. = $O(V+E)$

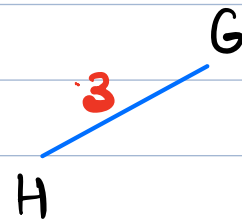


Dijkstra's Algorithm



A	B	C	D	E	F	G	H
0	2	9	3	5	4	10	13

MinHeap $\langle \text{dist}, \text{Node} \rangle$



~~$\langle 0, A \rangle$~~

~~$\langle 3, D \rangle$~~

~~$\langle 2, B \rangle$~~

~~$\langle 6, E \rangle$~~

~~$\langle 4, F \rangle$~~

~~$\langle 11, C \rangle$~~

~~$\langle 5, E \rangle$~~

~~$\langle 9, C \rangle$~~

~~$\langle 10, G \rangle$~~

~~$\langle 13, H \rangle$~~

MinHeap $\begin{cases} \text{insert}(x) \\ \text{extractMin}() \end{cases} \log(N)$
 $\Downarrow$
 Syntax in Java.

$$\begin{aligned} \langle 0, A \rangle &= d_A + w_{A \rightarrow N} \\ \rightarrow D &= 0 + 3 = 3 \\ \rightarrow B &= 0 + 2 = 2 \\ \rightarrow E &= 0 + 6 = 6 \\ \rightarrow F &= 0 + 4 = 4 \end{aligned}$$

$\langle 2, B \rangle$

$$\rightarrow C = 2 + 9 = 11$$

$\langle 4, F \rangle$

$$\begin{aligned} &\rightarrow A = 4 + 4 = 8 \\ &\rightarrow E = 4 + 5 = 9 \\ &\rightarrow G = 4 + 6 = 10 \end{aligned}$$

$$\begin{aligned} &\rightarrow E = 2 + 3 = 5 \\ &\rightarrow A = 2 + 2 = 4 \end{aligned}$$

$\langle 3, D \rangle$

$$\begin{aligned} &\rightarrow C = 3 + 6 = 9 \\ &\rightarrow A = 3 + 3 = 6 \end{aligned}$$

$\langle 5, E \rangle$

$$\begin{aligned} &\rightarrow A = 5 + 6 = 11 \\ &\rightarrow B = 5 + 3 = 8 \\ &\rightarrow F = 5 + 5 = 10 \end{aligned}$$

~~$\langle 6, E \rangle$~~

$\langle 9, C \rangle$

$$\begin{aligned} &\rightarrow B \quad \times \\ &\rightarrow D \quad \times \end{aligned}$$

$\langle 10, G \rangle$

$$\begin{aligned} &\rightarrow H = 10 + 3 = 13 \\ &\rightarrow F = 10 + 6 = 16 \end{aligned}$$

~~$\langle 11, C \rangle$~~

$\langle 13, H \rangle$

$$\rightarrow G = 13 + 3 = 16 \quad \times$$

Code

```
MinHeap < Pair < int, Node > > mhi;  
HashMap < Node, int > ans;
```

```
for (all Nodes u) d  
    ans.put (u, ∞);  
}
```

```
ans.update (src, 0);  
mh.insert (new Pair (0, src));
```

```
while (! mh.isEmpty()) {
```

```
    Pair p = mh.extractMin();  
    d = p.first();  
    Node u = p.second();
```

```
    if (d == ans.get(u)) {
```

```
        for (all Node v connected to u) {
```

```
             $d_v = d + W_{u-v};$ 
```

```
            if ( $d_v < ans.get(v)$ ) {
```

```
                ans.update (v,  $d_v$ );  
                mh.insert (new Pair ( $d_v$ , v));
```

```
            }
```

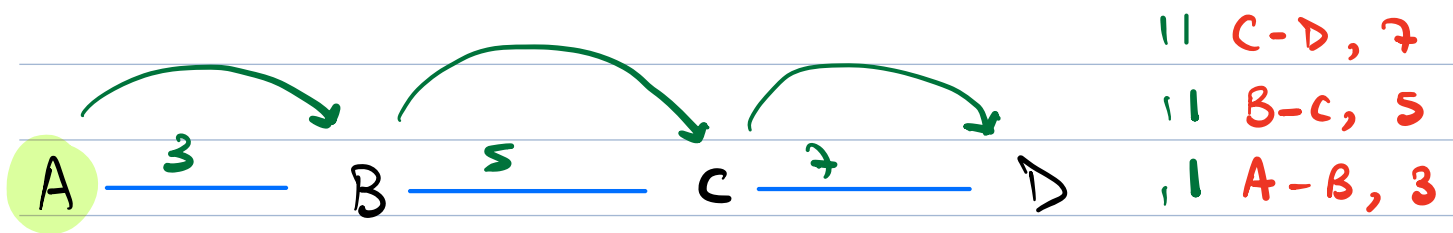
```
        }
```

```
    }
```

```
}
```

Bellman ford Algorithm

Relax all edges $V-1$ times.



A	B	C	D
0	3	8	15

I

II

III

C → D

$$\infty + 7 = \infty$$

$$\infty + 7 = 8 \times$$

$$8 + 7 = \underline{15}$$

B → C

$$\infty + 5 = \infty$$

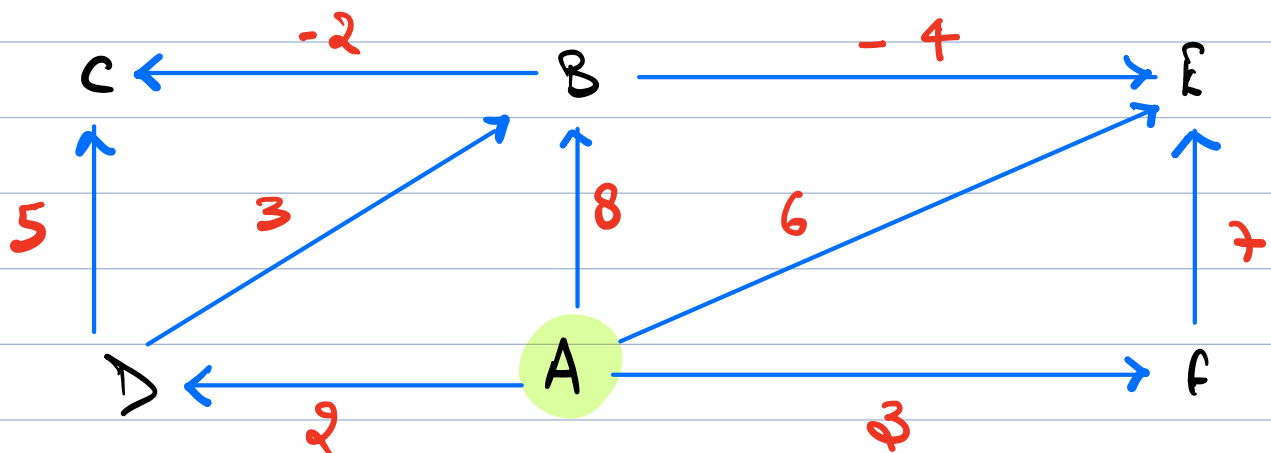
$$3 + 5 = 8$$

A → B

$$0 + 3 = 3$$

$$0 + 3 = 3$$

Dry Run



A	B	C	D	E	F
0	5	3	2	1	3

$$\times \quad A - B, 8$$

$$0 + 8 = 8$$

$$\times \quad A - E, 6$$

$$0 + 6 = 6$$

$$\times \quad A - D, 2$$

$$0 + 2 = 2$$

$$\times \quad B - C, -2$$

$$5 + (-2) = \cancel{3}$$

$$\times \quad B - E, -4$$

$$\cancel{5} + (-4) = \cancel{1}$$

$$\times \quad D - B, 3$$

$$2 + 3 = 5$$

$$\times \quad F - E, 7$$

$$\cancel{3} + 7 = 10$$

$$\times \quad A - F, 3$$

$$0 + 3 = 3$$

$$\times \quad D - C, 5$$

$$2 + 5 = 7$$

Routing Protocol Families

➤ Distance Vector Routing (Bellman Ford)

How it works:

Each router knows costs to its **direct neighbors**

Periodically broadcasts its **entire routing table** to neighbors

Neighbors update their tables based on received info

Converges over many rounds

Problem: Count to infinity.

Protocols: RIP, BGP

2) Link State Routing (Dijkstra)

How it works:

Each router floods its **local link state** (neighbors + costs) to ALL routers

Every router builds a **complete topology map**

Every router independently runs **Dijkstra** to compute shortest paths

Protocols: OSPF, IS-IS